

RELATÓRIO TÉCNICO: FRAMEWORK DE QUIZ

Disciplina: Princípios e Padrões de Projetos (BCC FACOM UFU)

Profa. Fabíola S. F. Pereira

1. Visão Geral Arquitetural

O projeto apresenta um framework orientado a objetos, implementado em Java, destinado à construção de aplicações de questionários (quizzes). A arquitetura foi rigorosamente dividida em dois pacotes conceituais principais: o `quiz.framework`, que fornece a infraestrutura genérica e independente de interfaces gráficas, e o `quiz.aplicacao`, que contém as particularidades de cada domínio e implementações de apresentação (Swing e Console). Esta separação garante o princípio de inversão de dependência e mantém um baixo acoplamento.

2. Padrões de Projeto Aplicados

O design do framework utilizou padrões do GoF (Gang of Four) para promover alta coesão, extensibilidade e reutilização de código:

- **Template Method:** Implementado na classe abstrata `FluxoQuizTemplate`. Ele define o esqueleto do algoritmo de execução do jogo (inicialização, iteração de perguntas, verificação de respostas, cálculo de pontos e exibição de feedback). As aplicações clientes herdam esta classe e implementam exclusivamente o método abstrato `configurarDependencias()`, mantendo o controle centralizado do fluxo no framework.
- **Strategy:** O cálculo de pontuação foi abstraído na interface `EstrategiaPontuacao`. Isso permite plugar dinamicamente diferentes regras de negócios sem alterar a lógica principal. Foram implementadas as estratégias concretas `PontuacaoAcertoSimples` (apenas adição de pontos) e `PontuacaoComPenalidade` (subtração de pontos por erro).
- **Abstract Factory:** A interface `QuizFactory` encapsula a criação da família de objetos necessários para uma aplicação cliente: a interface de exibição (View), a estratégia de pontuação e o gerenciador de perguntas. As fábricas concretas (`FabricaQuizLogica` e `FabricaQuizProgramacao`) garantem que as dependências sejam criadas com integridade mútua.
- **Observer (Bônus):** O padrão foi aplicado para viabilizar um sistema de múltiplos jogadores. A interface `QuizObserver` possibilita que classes externas acompanhem os eventos do quiz. A classe `RankingObserver` escuta os acertos/erros para atualizar assincronamente a pontuação num `RankingRepository` compartilhado pelo `LobbyMultiplayer`.

3. Aplicações Clientes Desenvolvidas

Para atestar a robustez e a flexibilidade do framework, duas aplicações concretas foram criadas consumindo exatamente a mesma base arquitetural:

- **Quiz de Lógica Proposicional:** Executado integralmente via terminal. Utiliza `ConsoleQuizView` (que implementa `QuizView`) e pune os jogadores por erros utilizando a estratégia de pontuação com penalidade.
- **Quiz de Programação (Multiplayer):** Apresenta uma Interface Gráfica interativa desenvolvida com *Java Swing* (`SwingQuizView`). Conta com um lobby para múltiplos jogadores que se alternam no mesmo ambiente de forma síncrona. Em caso de empate na pontuação final, invoca rodadas de desempate automaticamente, rastreadas pelos *Observers*.

4. Conclusão

O design empregado respeitou a restrição principal: o código do framework não necessita de nenhuma alteração para suportar novos domínios ou interfaces. Através de interfaces bem definidas (`QuizView`, `EstrategiaPontuacao`) e padrões clássicos da orientação a objetos, alcançou-se uma arquitetura extensível. A inclusão opcional do padrão *Observer* abriu o leque da ferramenta, tornando-a capaz de suportar regras sistêmicas globais (Rankings Multiplayer) de forma transparente para a lógica do quiz.